

13. Eager Loading Associations trong Spring Boot (JPA/Hibernate)

13.1. Giới thiệu về Eager Loading

Trong JPA (Java Persistence API) và Hibernate, việc tải dữ liệu liên quan (associations) có thể được thực hiện theo hai chiến lược chính: Lazy Loading (tải lười biếng) và Eager Loading (tải ngay lập tức).

- **Lazy Loading (Tải lười biếng):** Đây là chiến lược mặc định cho các mối quan hệ `OneToMany` và `ManyToMany`. Dữ liệu liên quan sẽ không được tải từ cơ sở dữ liệu cho đến khi bạn thực sự truy cập vào nó (ví dụ: gọi `getBooks()` trên một đối tượng `Author`). Lazy Loading giúp tiết kiệm tài nguyên bằng cách chỉ tải những gì cần thiết, nhưng có thể dẫn đến vấn đề N+1 Query nếu không được quản lý đúng cách.
- **Eager Loading (Tải ngay lập tức):** Đây là chiến lược mặc định cho các mối quan hệ `ManyToOne` và `OneToOne`. Dữ liệu liên quan sẽ được tải cùng với thực thể chính ngay lập tức khi thực thể chính được truy vấn. Eager Loading đảm bảo rằng bạn có sẵn dữ liệu liên quan ngay lập tức, nhưng có thể dẫn đến việc tải quá nhiều dữ liệu không cần thiết nếu không được sử dụng cẩn thận.

Trong tài liệu này, chúng ta sẽ tập trung vào Eager Loading, các trường hợp sử dụng, ưu nhược điểm và cách quản lý nó một cách hiệu quả.

13.2. Cấu hình Eager Loading

Bạn có thể cấu hình chiến lược tải dữ liệu bằng cách sử dụng thuộc tính `fetch` trong các annotation mối quan hệ của JPA.

Ví dụ:

Giả sử chúng ta có hai thực thể `Author` và `Book` với mối quan hệ One-to-Many (một tác giả có nhiều sách) và `Book` có mối quan hệ Many-to-One với `Author`.

```
Java
```

```

import javax.persistence.*;
import java.util.List;

@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    // Mặc định là FetchType.LAZY cho OneToMany
    @OneToMany(mappedBy = "author", fetch = FetchType.EAGER) // Thay đổi
    // thành EAGER
    private List<Book> books;

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public List<Book> getBooks() { return books; }
    public void setBooks(List<Book> books) { this.books = books; }
}

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    // Mặc định là FetchType.EAGER choManyToOne
    @ManyToOne(fetch = FetchType.EAGER) // Có thể để mặc định hoặc chỉ định
    // rõ EAGER
    @JoinColumn(name = "author_id")
    private Author author;

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public Author getAuthor() { return author; }
    public void setAuthor(Author author) { this.author = author; }
}

```

Trong ví dụ trên:

- Mỗi quan hệ `Author` -> `Book` (`@OneToMany`) được đặt là `FetchType.EAGER` . Điều này có nghĩa là khi bạn tải một đối tượng `Author` , tất cả các đối tượng `Book` liên quan cũng sẽ được tải ngay lập tức trong cùng một truy vấn hoặc các truy vấn bổ sung ngay sau đó.
- Mỗi quan hệ `Book` -> `Author` (`@ManyToOne`) được đặt là `FetchType.EAGER` (hoặc có thể bỏ qua vì đây là mặc định). Khi bạn tải một đối tượng `Book` , đối tượng `Author` liên quan cũng sẽ được tải ngay lập tức.

13.3. Ưu và nhược điểm của Eager Loading

13.3.1. Ưu điểm

- **Dễ sử dụng:** Dữ liệu liên quan luôn có sẵn ngay lập tức khi bạn truy cập thực thể chính, không cần lo lắng về `LazyInitializationException` .
- **Giảm số lượng truy vấn (trong một số trường hợp):** Đối với các mối quan hệ `ManyToOne` và `OneToOne` , Eager Loading thường được thực hiện bằng cách sử dụng `JOIN` trong truy vấn ban đầu, giúp giảm số lượng truy vấn so với Lazy Loading (nếu bạn luôn cần dữ liệu liên quan).

13.3.2. Nhược điểm

- **Tải quá nhiều dữ liệu (Over-fetching):** Đây là nhược điểm lớn nhất. Eager Loading sẽ tải tất cả dữ liệu liên quan, ngay cả khi bạn không cần chúng trong một trường hợp sử dụng cụ thể. Điều này có thể dẫn đến:
 - **Giảm hiệu suất:** Tốn thời gian hơn để truy vấn và truyền tải dữ liệu từ database.
 - **Tăng bộ nhớ sử dụng:** Dữ liệu không cần thiết được tải vào bộ nhớ ứng dụng.
- **Vấn đề N+1 Query (đối với `OneToMany` / `ManyToMany`):** Mặc dù bạn đặt `FetchType.EAGER` cho `OneToMany` hoặc `ManyToMany` , Hibernate vẫn có thể thực hiện N+1 Query nếu không có các biện pháp tối ưu hóa khác (như `JOIN FETCH` hoặc `@BatchSize`). Điều này là do `EAGER` chỉ đảm bảo dữ liệu được tải, chứ không đảm bảo cách thức tải hiệu quả nhất.

- **Cartesian Product:** Khi có nhiều mối quan hệ `EAGER` trên cùng một thực thể, Hibernate có thể tạo ra một Cartesian Product, dẫn đến việc trả về số lượng bản ghi lớn hơn nhiều so với mong đợi, gây ra lỗi hoặc vấn đề hiệu suất.
- **Không linh hoạt:** Khó thay đổi chiến lược tải dữ liệu cho từng trường hợp sử dụng cụ thể.

13.4. Khi nào nên sử dụng Eager Loading?

Eager Loading nên được sử dụng một cách thận trọng và chỉ trong các trường hợp sau:

- **Mối quan hệ `ManyToOne` và `OneToOne`:** Đây là các mối quan hệ mà `FetchType.EAGER` là mặc định và thường an toàn để sử dụng, đặc biệt khi bạn chắc chắn rằng dữ liệu liên quan luôn cần thiết. Ví dụ: một `Order` luôn cần thông tin `Customer`.
- **Khi bạn chắc chắn luôn cần dữ liệu liên quan:** Nếu một mối quan hệ luôn được truy cập mỗi khi thực thể chính được tải, Eager Loading có thể hợp lý.
- **Số lượng dữ liệu liên quan nhỏ và cố định:** Nếu mối quan hệ `OneToMany` hoặc `ManyToMany` chỉ có một số lượng rất nhỏ các thực thể con và không thay đổi nhiều, Eager Loading có thể chấp nhận được.

Trong hầu hết các trường hợp, đặc biệt là với các mối quan hệ `OneToMany` và `ManyToMany`, nên ưu tiên Lazy Loading và sử dụng các kỹ thuật tối ưu hóa như `JOIN FETCH`, `@EntityGraph`, hoặc `@BatchSize` để tải dữ liệu một cách hiệu quả khi cần.

13.5. Eager Loading và N+1 Query Problem

Như đã đề cập trong tài liệu "12. Vấn đề N+1 Query", việc đặt `FetchType.EAGER` cho mối quan hệ `OneToMany` hoặc `ManyToMany` **không tự động giải quyết vấn đề N+1 Query**. Hibernate vẫn có thể thực hiện N+1 Query nếu không có các biện pháp tối ưu hóa khác.

Ví dụ:

Nếu bạn có `Author` với `List<Book>` được đặt là `FetchType.EAGER` và bạn gọi `authorRepository.findAll()`, Hibernate sẽ thực hiện:

1. Một truy vấn để lấy tất cả các `Author` .
2. N truy vấn riêng biệt để lấy `Book` cho từng `Author` .

Để thực sự giải quyết N+1 Query khi bạn muốn tải dữ liệu liên quan một cách eager, bạn cần sử dụng các kỹ thuật sau:

- **JOIN FETCH (khuyến nghị):** Sử dụng trong JPQL/HQL để tải dữ liệu liên quan trong một truy vấn duy nhất.
- **@EntityGraph** : Cung cấp một cách khai báo để thực hiện `JOIN FETCH` .
- **@BatchSize** : Giúp giảm số lượng truy vấn bằng cách tải dữ liệu theo lô.
- **FetchMode.SUBSELECT** : Tải dữ liệu liên quan bằng một truy vấn con.

13.6. Bản chất của Eager Loading Associations

Bản chất của Eager Loading Associations trong JPA/Hibernate là một chiến lược tải dữ liệu mà theo đó, các dữ liệu liên quan sẽ được tải ngay lập tức cùng với thực thể chính. Mặc dù có vẻ tiện lợi vì dữ liệu luôn sẵn có, nhưng việc sử dụng Eager Loading một cách không cẩn thận, đặc biệt với các mối quan hệ `OneToMany` và `ManyToMany` , có thể dẫn đến các vấn đề nghiêm trọng về hiệu suất như tải quá nhiều dữ liệu không cần thiết, tăng mức sử dụng bộ nhớ, và thậm chí gây ra vấn đề N+1 Query. Do đó, việc hiểu rõ bản chất và các nhược điểm của Eager Loading là rất quan trọng để đưa ra quyết định đúng đắn về chiến lược tải dữ liệu, ưu tiên Lazy Loading và sử dụng các kỹ thuật tối ưu hóa truy vấn khi cần thiết để đạt được hiệu suất tối ưu cho ứng dụng.